# Project Report

Medical Diagnosis Rule-Based Expert System

## 1. Introduction & Objectives

This report documents the design and implementation of a simple rule-based expert system tailored for medical diagnostics. The system evaluates user-provided symptoms and leverages an inference engine to deduce the most probable illness.

The primary objectives of this project are:

- To demonstrate foundational Knowledge Representation techniques using Python.
- To implement a subset-matching inference engine capable of processing strict IF-THEN rules.
- To design and document Semantic Networks mapping the relationships between patients, symptoms, and medical conditions.

## 2. Knowledge Base Specification

The system operates on a localized knowledge base containing discrete lists of known symptoms and diagnosable diseases.

> **Known Symptoms:** Fever, Headache, Cough, Chest Pain, Sneezing, Runny Nose, Fatigue, Sore Throat, Vomiting, Diarrhea.
>
> **Diagnosable Conditions:** Malaria, Pneumonia, Flu, Food Poisoning.

### 2.1 Inference Rules

The engine uses forward-chaining logic. The rules are structured conditionally as follows:

| Rule ID | Conditions (IF user has...) | Conclusion (THEN disease is...) |
|---|---|---|
| A1 | Fever, Headache, Fatigue | Malaria |
| A2 | Cough, Chest Pain, Fatigue | Pneumonia |
| A3 | Sneezing, Runny Nose, Sore Throat | Flu |
| A4 | Vomiting, Diarrhea, Fatigue | Food Poisoning |

## 3. Semantic Network Topologies

To accurately model the logical flow of the system, two distinct semantic network topologies were developed.

### 3.1 Patient-Centric (Hub-and-Spoke) Model

This model places the Patient at the absolute center of the graph. Relationships radiate outward via *"exhibits"* edges to the respective symptoms. The symptoms then funnel into diagnoses via *"indicates"* edges. This accurately represents the sequential user-flow of the application.

### 3.2 Hierarchical Ontology Model

This classical approach utilizes strict *"is-a"* and *"has-symptom"* relationships. The parent classes (Disease and Symptom) sit at the top. Specific instances (e.g., Malaria, Fever) branch below them, visually mapping how the Python dictionary structures categorizations in memory. It perfectly illustrates node-sharing (e.g., Fatigue being mapped to multiple conditions).

## 4. System Implementation & Bonuses

The core application was developed in Python 3. The separation of concerns was strictly maintained by decoupling the data (`knowledge_base.py`) from the operational logic (`main.py`).

As part of the **Bonus Activity** requirements, input sanitization and validation were integrated. The system parses the user's comma-separated string, converts it to title case, and cross-

references it against the internal symptom list. Any unrecognized syntax or spelling error prompts an informative retry loop, ensuring the inference engine is never fed faulty data.

## 5. Conclusion

The resulting prototype successfully meets all assignment criteria. By combining a clean, dictionary-based knowledge representation with strict subset-matching logic, the system effectively acts as a rudimentary AI diagnostician. The accompanying semantic network diagrams successfully bridge the gap between abstract code and visual logic models.